

## HCL Python Sprint Challenge

### Build an ATM Simulator

#### Problem Statement:

Automated Teller Machines (ATMs) are widely used for essential banking operations such as withdrawing money, checking balances, and depositing funds. Behind every ATM is a system that securely verifies users and processes transactions accurately.

Your challenge is to build a Python-based ATM Simulator that replicates the core functionalities of a real ATM. The system should be reliable, user-friendly, and logically structured.

This is a rapid coding challenge intended to test your programming fundamentals, logical thinking, and ability to design structured solutions under time constraints.

#### Objective:

Develop a Python program that allows a user to perform essential banking operations after successful authentication. Your simulator must behave like a real ATM as closely as possible within a command-line interface.

## Mandatory Features:

### 1. User Authentication

- The system should require a PIN to access the account.
- Allow a maximum of three incorrect attempts.
- Exit the program after three failed tries.

### 2. Menu-Driven Interface

After login, display options such as:

- Check Balance
- Deposit Money
- Withdraw Money
- Exit

The program should continue running until the user chooses to exit.

### 3. Balance Management

- Maintain an account balance.
- Update it correctly after every transaction.

### 4. Withdrawal Rules

- Prevent withdrawal if funds are insufficient.
- Display a clear message in such cases.

### 5. Deposit Function

- Accept deposit amounts.
- Update the balance immediately.

## 6. Clean Output

All messages should be clear and user-friendly.

Examples:

- "Transaction Successful."
- "Insufficient Balance."
- "Please enter a valid amount."

Technical Constraints:

- Use only Python fundamentals.
- Do not use external APIs.
- Do not use databases.
- The program must run without errors.

### Challenge Features (For Competitive Advantage):

Teams should be implementing any 3 the following in addition to the above features:

- Transaction history
- Multiple user accounts
- PIN change functionality
- Account lock after failed attempts
- Receipt generation
- Error handling for invalid inputs
- Object-Oriented Programming approach
- Simple graphical interface (optional)

### Deliverables:

Each team must present:

- A working Python program
- Explanation of the program logic
- Live demonstration
- Key design choices

Presentation Time: 7–10 minutes per team.

### Evaluation Criteria:

Teams will be evaluated based on:

- Logical correctness
- Code structure and readability
- Handling of edge cases
- Program stability
- Feature completeness
- Presentation clarity

In the event of similar solutions, preference will be given to programs that are cleaner, more structured, and more user-friendly.

### Outcome Expectation:

This challenge is designed to strengthen your programming discipline and simulate real-world system design at a foundational level. Focus on writing efficient, logical, and maintainable code rather than unnecessarily complex implementations.